

Laporan Proyek
Pengembangan Sistem Analitik Penjualan E-Commerce
Berbasis Big Data Menggunakan Apache Spark dengan
Integrasi SQL dan NoSQL



11423013	Esticka Priscila Sibarani
11423017	Anisetus Bambang Manalu
11423022	Wahyudi G. Lumbantoruan
11423036	Gabriel Domingo A. Siregar
11423039	Nicolas Prasetia H. Batubara

D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
2024/2025

Daftar Isi

Daftar Isi	2
Daftar Tabel	4
BAB I PENDAHULUAN	5
1.1. Latar Belakang	5
1.2. Rumusan Masalah	5
1.3. Tujuan	6
1.4. Manfaat Penelitian	6
BAB II TINJAUAN PUSTAKA	7
2.1 Landasan Teori	7
2.1.1 Big Data dan Ekosistem Hadoop/Spark	7
2.1.2 Apache Spark dan PySpark	7
2.1.3 MySQL sebagai Basis Data Relasional	7
2.1.4 MongoDB sebagai Basis Data NoSQL	7
2.1.5 E-Commerce Analytics	8
2.1.6 ETL Pipeline dan Data Engineering	8
2.1.7 Penelitian Terdahulu	8
BAB III DESKRIPSI DATASET	9
3.1 Deskripsi Data	9
3.2 Jumlah dan Karakteristik Data	9
3.3 Variabel Penelitian	9
3.4 Struktur Dataset	10
BAB IV METODOLOGI DATA ENGINEERING	12
4.1 Kerangka Kerja Pipeline (Adaptasi CRISP-DM)	12
4.2 Sumber Data	12
4.3 Arsitektur Sistem	13
4.4 Konfigurasi Lingkungan	13
4.6 Strategi Penyimpanan (MySQL vs MongoDB)	14
BAB V EDA (Exploratory Data Analysis) & Insight	16
5.1 Statistik Deskriptif Dataset	16
5.2 Analisis Tren Bulanan	16
5.3 Analisis per Kategori Produk	16
5.4 Analisis Distribusi Status Order	17
5.5 Analisis Distribusi Status Order	17
5.6 Analisis Distribusi Status Order	17

5.7 Analisis Distribusi Status Order	18
BAB VI IMPLEMENTASI PIPELINE DAN HASIL.....	19
6.1 Inisialisasi PySpark Sessions.....	19
6.2 Data Ingestion.....	19
6.3 Data Cleaning dan Feature Engineering	20
6.4 Analisis Agregat dengan Spark SQL.....	20
6.5 Penyimpanan ke MySQL	21
6.6 Penyimpanan ke MongoDB	21
6.7 Integrasi Cross-System (JOIN MySQL x MongoDB)	22
6.8 Dashboard Streamlit	22
6.9 Ringkasan Hasil	22
BAB VII KESIMPULAN	24
7.1 Kesimpulan	24
LAMPIRAN	25

Daftar Tabel

Tabel 1. Jumlah dan Kategori Data	9
Tabel 2. Variabel Penelitian	9
Tabel 3. Struktur Dataset	10
Tabel 4. Sumber Data	13
Tabel 5. Konfigurasi Lingkungan	13
Tabel 6. MySQL vs MongoDB	14
Tabel 7. Statistik Deskriptif Dataset	16
Tabel 8. Analisis Distribusi Status Order	17
Tabel 9. Inisialisasi PySpark Sessions	19
Tabel 10. Data Cleaning dan Feature Engineering	20
Tabel 11. Penyimpanan ke MySQL	21
Tabel 12. Penyimpanan ke MongoDB	21
Tabel 13. Integrasi Cross-System	22
Tabel 14. Ringkasan Hasil	22

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi digital telah mendorong pertumbuhan platform e-commerce secara pesat di berbagai negara. Amazon sebagai salah satu platform e-commerce terbesar di dunia memproses jutaan transaksi setiap hari sehingga menghasilkan data dalam jumlah yang sangat besar. Data transaksi tersebut menyimpan berbagai informasi penting yang dapat dimanfaatkan untuk mendukung pengambilan keputusan bisnis secara lebih efektif.

Dalam kondisi tersebut, pengelolaan dan analisis Big Data menjadi salah satu faktor penting dalam menciptakan keunggulan kompetitif. Dataset Amazon Sale Report yang digunakan pada proyek ini terdiri dari 128.976 data transaksi dengan 24 atribut, yang mencakup informasi terkait status pesanan, kategori produk, hingga detail pengiriman di berbagai wilayah.

Proyek ini membangun alur pengolahan data secara lengkap menggunakan beberapa teknologi, yaitu Apache Spark (PySpark) untuk memproses data, MySQL untuk menyimpan data terstruktur, MongoDB untuk menyimpan data semi-terstruktur, serta Streamlit untuk menampilkan dashboard visualisasi interaktif. Sistem ini dirancang untuk mengolah data mentah dari file CSV hingga menghasilkan informasi yang dapat digunakan untuk mendukung kebutuhan bisnis.

1.2. Rumusan Masalah

1. Bagaimana membangun pipeline Big Data yang efisien untuk memproses dataset Amazon Sale Report berskala 128.976 baris?
2. Bagaimana strategi penyimpanan data yang optimal antara MySQL (relasional) dan MongoDB (NoSQL) untuk kebutuhan analitik yang berbeda?
3. Bagaimana melakukan integrasi lintas sistem (cross-system JOIN) antara MySQL dan MongoDB menggunakan Spark DataFrame?
4. Bagaimana menyajikan insight bisnis dari data e-commerce Amazon secara interaktif melalui dashboard Streamlit?
5. Apa saja pola dan tren bisnis yang dapat ditemukan dari data penjualan Amazon tersebut?

1.3. Tujuan

Tujuan dari penelitian ini adalah untuk:

1. Membangun pipeline Big Data end-to-end menggunakan PySpark untuk memproses data Amazon Sale Report dari ingestion hingga analisis
2. Merancang dan mengimplementasikan strategi penyimpanan hybrid menggunakan MySQL untuk data terstruktur dan MongoDB untuk data semi-terstruktur
3. Mengimplementasikan integrasi cross-system antara MySQL dan MongoDB melalui Spark DataFrame JOIN tanpa menggunakan Pandas sebagai perantara
4. Membangun dashboard Streamlit interaktif untuk visualisasi insight bisnis dari pipeline yang dibangun
5. Mengidentifikasi pola penjualan, tren bulanan, distribusi kategori, dan segmentasi harga dari dataset Amazon.

1.4. Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut:

1. Memberikan pemahaman praktis tentang implementasi Big Data Pipeline menggunakan teknologi modern (Spark, MySQL, MongoDB)
2. Menyediakan template arsitektur pipeline yang dapat diadaptasi untuk kebutuhan analitik e-commerce lainnya
3. Menghasilkan insight bisnis actionable dari data penjualan Amazon yang dapat digunakan untuk pengambilan keputusan
4. Mendemonstrasikan integrasi multi-database (SQL dan NoSQL) dalam satu ekosistem analitik yang terintegrasi

BAB II

TINJAUAN PUSTAKA

2.1 Landasan Teori

2.1.1 Big Data dan Ekosistem Hadoop/Spark

Big Data adalah kumpulan data yang memiliki volume, velocity, dan variety yang sangat besar sehingga tidak dapat diproses secara efisien menggunakan sistem basis data yang biasa. Ekosistem Hadoop menyediakan framework terdistribusi untuk penyimpanan (HDFS) dan pemrosesan (MapReduce) data berskala besar. Apache Spark hadir sebagai pengembangan dari MapReduce dengan kemampuan pemrosesan data menggunakan in-memory yang jauh lebih cepat.

2.1.2 Apache Spark dan PySpark

Apache Spark adalah framework open-source yang digunakan untuk memproses data dalam jumlah besar secara terdistribusi. Spark dapat digunakan untuk pengolahan data batch, streaming, machine learning, dan analisis data. PySpark merupakan versi Spark yang menggunakan bahasa Python sehingga lebih mudah digunakan oleh pengembang. Pada proyek ini, PySpark digunakan untuk ingestion, cleaning, transformasi, dan analisis data, serta untuk mengelola distribusi beban kerja pada mode local.

2.1.3 MySQL sebagai Basis Data Relasional

MySQL adalah sistem manajemen basis data relasional yang menggunakan bahasa SQL (RDBMS). MySQL cocok digunakan untuk menyimpan data yang terstruktur karena memiliki schema yang jelas serta mendukung proses query dan transaksi dengan baik. Dalam proyek ini, MySQL digunakan untuk menyimpan data seperti orders, monthly_summary, dan category_stats yang membutuhkan proses analisis data yang lebih kompleks.

2.1.4 MongoDB sebagai Basis Data NoSQL

MongoDB adalah basis data NoSQL yang menyimpan data dalam bentuk dokumen dengan format BSON atau Binary JSON. MongoDB lebih fleksibel untuk menyimpan data semi-terstruktur dan data dengan struktur yang tidak tetap. Pada proyek ini, MongoDB digunakan untuk menyimpan data detail pesanan (orders_detail) dan hasil analisis kategori produk (category_insights) yang memiliki struktur data lebih kompleks, seperti array dan nested object.

2.1.5 E-Commerce Analytics

Analitik e-commerce adalah proses mengumpulkan dan menganalisis data dari aktivitas perdagangan elektronik. Beberapa data yang biasanya dianalisis meliputi total pendapatan, jumlah pesanan yang berhasil dikirim, kategori produk yang paling banyak terjual, persebaran penjualan berdasarkan wilayah, hingga tren penjualan dalam periode tertentu. Analisis ini membantu perusahaan dalam menentukan strategi pemasaran, mengatur stok barang, dan meningkatkan pelayanan kepada pelanggan.

2.1.6 ETL Pipeline dan Data Engineering

ETL (Extract, Transform, Load) merupakan proses penting dalam data engineering yang terdiri dari pengambilan data dari sumber, pembersihan serta pengolahan data, kemudian penyimpanan data ke sistem tujuan. Pada sistem Big Data, proses ini juga sering disebut ELT karena data terlebih dahulu disimpan, lalu diproses menggunakan kemampuan komputasi dari sistem Big Data.

2.1.7 Penelitian Terdahulu

Beberapa penelitian sebelumnya telah membahas analisis data e-commerce dengan berbagai metode. Penelitian tersebut umumnya berfokus pada pencarian pola pembelian pelanggan, segmentasi pelanggan, dan prediksi kebutuhan produk. Selain itu, penggunaan kombinasi basis data SQL dan NoSQL juga dinilai efektif untuk mengelola data e-commerce yang memiliki struktur beragam dan juga telah terbukti efektif dalam menangani heterogenitas data e-commerce modern.

BAB III

DESKRIPSI DATASET

3.1 Deskripsi Data

Dataset yang digunakan pada proyek ini adalah Amazon Sale Report, yaitu dataset transaksi penjualan dari platform Amazon India. Dataset disimpan dalam format CSV (Comma-Separated Values) dan berisi data transaksi e-commerce yang mencakup informasi pesanan, status pengiriman, detail produk, serta wilayah tujuan pengiriman.

Dataset ini diperoleh dari sumber publik dan digunakan untuk keperluan akademis sebagai contoh penerapan Big Data Pipeline. Data yang digunakan memiliki karakteristik seperti data pada dunia nyata, sehingga masih terdapat beberapa missing value, data yang tidak konsisten, serta perbedaan format penulisan tanggal.

3.2 Jumlah dan Karakteristik Data

Tabel 1. Jumlah dan Kategori Data

Karakteristik	Keterangan
Jumlah Baris	128.976 transaksi
Jumlah Kolom	24 kolom (termasuk 2 kolom yang tidak berguna)
Format File	CSV (Comma-Separated Values)
Ukuran File	65,7 MB
Periode Data	Transaksi e-commerce Amazon India
Mata Uang	INR (Indian Rupe)
Sumber Data	Amazon Sale Report (Publik)

3.3 Variabel Penelitian

Dataset memiliki 24 kolom yang dapat dikelompokkan berdasarkan fungsinya. Berikut nama serta penjelasan lengkap variabel yang digunakan, yaitu:

Tabel 2. Variabel Penelitian

Nama Kolom	Tipe Data	Keterangan
order_id	String	ID unik setiap order, Primary Key
date	String => Date	Tanggal order (format M/d/yyyy) diekstrak menjadi year, month, month_name
status	String	Status detail order (Shipped, Cancelled, Pending, dll.)
fulfilment	String	Tipe fulfillment (Amazon / Merchant)
sales_channel	String	Channel penjualan (Amazon.in)
ship_service_level	String	Level layanan pengiriman (Standard / Expedited)
sku	String	Stock Keeping Unit, kode internal produk Kategori produk (Set, Kurta, Western Dress, dll.)

Nama Kolom	Tipe Data	Keterangan
category	String	Kategori produk (Set, Kurta, Western Dress, dll.)
size	String	Ukuran produk (S, M, L, XL, XXL, dll.)
asin	String	Amazon Standard Identification Number
courier status	String	Status kurir (Shipped, Cancelled, dll.), banyak null
qty	Integer	Jumlah item dalam order
currency	String	Mata uang transaksi (INR)
amount	Double	Harga satuan produk dalam INR
ship city	String	Kota tujuan pengiriman
ship state	String	Negara bagian tujuan pengiriman
ship_postal_code	String	Kode pos tujuan pengiriman
ship country	String	Negara tujuan (IN)
promotion_ids	String	ID promosi yang digunakan banyak null, disimpan sebagai Array di MongoDB
b2b	Boolean	Flag apakah order adalah transaksi B2B (Business-to-Business)
fulfilled by	String	Pihak yang memenuhi order — banyak null
row_index	String	Index baris (tidak berguna, di-drop)
unused col	String	Kolom kosong (tidak berguna, di-drop)
style	String	Style produk (tidak disertakan dalam penyimpanan)

3.4 Struktur Dataset

Berikut kolom-kolom tambahan yang dihasilkan selama tahap cleaning dan feature engineering.

Tabel 3. Struktur Dataset

Kolom Baru	Dibuat Dari	Deskripsi
year	date	Tahun dari tanggal order
month	date	Bulan dalam angka (1-12)
month_name	date	Nama bulan (January, February, dll.)
status_group	status	Normalisasi status: Shipped / Cancelled / Pending / Other
revenue	qty × amount	Total pendapatan per order (qty dikali amount)
has_promotion	promotion_ids	Flag Boolean: True jika ada promosi, False jika tidak

Dari 24 kolom pada dataset asli, terdapat 3 kolom yang dihapus karena tidak memiliki nilai analisis, yaitu row_index yang hanya berfungsi sebagai indeks baris pada file CSV, unused_col karena tidak memiliki isi data, dan style karena informasinya sudah terwakili oleh kolom sku.

Selain itu, kolom `ship_service_level` hanya disimpan di MongoDB dan tidak dimasukkan ke dalam tabel MySQL.

BAB IV

METODOLOGI DATA ENGINEERING

4.1 Kerangka Kerja Pipeline (Adaptasi CRISP-DM)

Pipeline pada proyek ini menggunakan kerangka CRISP-DM (Cross-Industry Standard Process for Data Mining) yang disesuaikan dengan kebutuhan Big Data Engineering. Tahapan yang dilakukan, yaitu:

- a. Business Understanding
Memahami kebutuhan analisis pada data e-commerce Amazon, seperti tren penjualan, kategori produk terlaris, efisiensi pengiriman, serta perbedaan transaksi B2B dan B2C
- b. Data Understanding
Melakukan eksplorasi terhadap dataset Amazon Sale Report yang terdiri dari sekitar 128.976 baris dan 24 kolom, termasuk mengidentifikasi missing value dan data yang tidak konsisten
- c. Data Preparation
Melakukan proses pembersihan data, normalisasi, dan pembuatan atribut baru menggunakan PySpark. Pada tahap ini juga dilakukan penghapusan kolom yang tidak diperlukan
- d. Pipeline Implementation
Membangun alur pengolahan data mulai dari pengambilan data, proses cleaning, agregasi data, penyimpanan ke MySQL dan MongoDB, integrasi data, hingga export data hasil pengolahan
- e. Evaluation
Memastikan data pada MySQL dan MongoDB telah terintegrasi dengan baik serta memeriksa konsistensi data antar sistem penyimpanan
- f. Deployment
Menampilkan hasil pengolahan data melalui dashboard Streamlit yang membaca data dari folder `sssstreamlit_data/`.

4.2 Sumber Data

Data pada proyek ini berasal dari file CSV lokal bernama Amazon Sale Report.csv. File tersebut berisi data transaksi penjualan Amazon India dengan ukuran sekitar 65 MB. Proses pipeline dimulai dengan membaca file CSV menggunakan PySpark dengan schema yang telah

ditentukan secara eksplisit agar tipe data tetap konsisten dan proses pembacaan data menjadi lebih cepat pada dataset berukuran besar.

Tabel 4. Sumber Data

Aspek	Detail
Nama File	Amazon Sale Report.csv
Ukuran	65,7 MB
Jumlah Baris	128.976 transaksi
Format	CSV dengan header
Encoding	UTF-8
Schema	Eksplisit (StructType dengan 24 StructField)

4.3 Arsitektur Sistem

Arsitektur pipeline pada proyek ini terdiri dari beberapa lapisan yang saling terhubung untuk mendukung proses pengolahan dan analisis data.

- Lapisan Sumber Data
File CSV Amazon Sale Report sebagai sumber data mentah.
- Lapisan Pemrosesan
Apache Spark (PySpark) dalam mode local[*] dengan alokasi memori 4GB untuk driver, menangani semua transformasi dan analisis.
- Lapisan Penyimpanan Relasional
MySQL (localhost:3306) dengan database amazon_sales_db, menyimpan 3 tabel: orders, monthly_summary, category_stats.
- Lapisan Penyimpanan NoSQL
MongoDB (localhost:27017) dengan database amazon_sales, menyimpan 2 koleksi: orders_detail dan category_insights.
- Lapisan Integrasi
Cross-system JOIN menggunakan Spark DataFrame — MySQL dibaca via JDBC, MongoDB dibaca via PyMongo lalu dikonversi ke Spark DataFrame.
- Lapisan Visualisasi
Dashboard Streamlit (app.py) membaca data JSON hasil export pipeline dan menyajikannya dalam bentuk grafik interaktif.

4.4 Konfigurasi Lingkungan

Tabel 5. Konfigurasi Lingkungan

Komponen	Versi/Konfigurasi
Python	3.x
PySpark	Apache Spark (local[2], driver memory 4GB)
Java	JDK 17.0.11
Hadoop	3.3.6 (winutils untuk windows)
MySQL	localhost:3306, database: final_bidat

Komponen	Versi/Konfigurasi
MongoDB	localhost:27017, database: final_bidat
MySQL JDBC Driver	mysql-connector-j-9.7.0.jar
Streamlit	Dashboard (app.py)
Plotly	Visualisasi grafik interaktif

4.5 Pra-Pemrosesan Data (Data Cleaning)

Tahap pra-pemrosesan dilakukan menggunakan PySpark dan mencakup langkah-langkah berikut:

- Drop kolom tidak berguna
row_index, unused_col, style di-drop karena tidak memberikan nilai analitik.
- Parsing tanggal
Kolom date dikonversi dari string (M/d/yyyy) ke tipe Date menggunakan to_date() dengan format yang sesuai.
- Ekstraksi fitur temporal
Dari kolom date diekstrak year, month, dan month_name menggunakan fungsi year(), month(), dan date_format().
- Normalisasi status
Dari kolom date diekstrak year, month, dan month_name menggunakan fungsi year(), month(), dan date_format().
- Feature engineering revenue
Kolom revenue dihitung sebagai perkalian qty dan amount ($\text{revenue} = \text{qty} \times \text{amount}$).
- Feature engineering has_promotion
Kolom revenue dihitung sebagai perkalian qty dan amount ($\text{revenue} = \text{qty} \times \text{amount}$).
- Penanganan missing values
Baris dengan amount null atau qty null di-drop karena tidak dapat digunakan untuk analisis finansial.

4.6 Strategi Penyimpanan (MySQL vs MongoDB)

Pemisahan penyimpanan antara MySQL dan MongoDB didasarkan pada karakteristik data dan kebutuhan query:

Tabel 6. MySQL vs MongoDB

Aspek	MySQL	MongoDB
Tipe Data	Flat/Tabular (normalized)	Nested/Semi-structured
Kegunaan Utama	Transaksi, JOIN, Reporting	Dokumen lengkap, Analytics
Tabel/koleksi	orders, monthly summary, category stats	order detail, category insights
Keunggulan	ACID compliance, Relasi, Indexing efisien	Schema fleksibel, Nested data, Array

Aspek	MySQL	MongoDB
Contoh Data Khusus	Data flat, agregat bulanan	promotion_ids sebagai Array, nested price range

BAB V

EDA (Exploratory Data Analysis) & Insight

5.1 Statistik Deskriptif Dataset

Setelah proses cleaning selesai dilakukan, dataset yang telah bersih (`df_clean`) digunakan untuk proses analisis. Statistik deskriptif memberikan gambaran umum mengenai kondisi data, baik dari sisi data numerik maupun kategorinya pada dataset Amazon Sale Report.

Tabel 7. Statistik Deskriptif Dataset

Metrik	Nilai
Total order diproses	128.976 transaksi
Total revenue (INR)	Ratusan juta INR
Order berhasil dikirim (Shipped)	70-75% dari total order
Jumlah kategori produk	Belasan kategori unik
Rentang harga produkss	Ratusan hingga ribuan INR
Kategori terlaris	Set (Pakaian kumpulan)

5.2 Analisis Tren Bulanan

Analisis tren bulanan dilakukan menggunakan fungsi `groupBy()` pada PySpark dengan memanfaatkan kolom `year` dan `month`. Proses ini menghasilkan informasi berupa total order, total revenue, jumlah produk terjual, serta jumlah kategori unik pada setiap bulan. Hasil analisis kemudian disimpan pada tabel `monthly_summary` di MySQL dan diekspor ke file `monthly_summary.json` untuk digunakan pada dashboard Streamlit.

Berdasarkan hasil analisis, terlihat adanya perubahan jumlah order dan revenue pada beberapa bulan tertentu. Perubahan tersebut kemungkinan dipengaruhi oleh musim belanja, promosi, atau faktor lainnya. Selain itu, dilakukan juga perhitungan pertumbuhan revenue bulanan atau Month-over-Month (MoM) menggunakan fungsi `LAG()` pada Spark untuk melihat persentase perubahan revenue antar bulan secara berurutan.

5.3 Analisis per Kategori Produk

Analisis kategori produk dilakukan menggunakan fungsi `groupBy("category")` pada PySpark untuk menghitung total order, jumlah produk terjual, total revenue, rata-rata harga, serta harga minimum dan maksimum pada setiap kategori produk. Hasil analisis tersebut disimpan pada

tabel `category_stats` di MySQL dan koleksi `category_insights` di MongoDB. Pada MongoDB, data juga dilengkapi dengan informasi tambahan seperti `price_range` dan `sizes_breakdown`. Hasil analisis menunjukkan bahwa kategori dengan revenue tertinggi didominasi oleh produk pakaian wanita India, seperti kategori *Set* dan *Kurta*. Hal ini menunjukkan bahwa produk fashion memiliki kontribusi besar terhadap penjualan di platform Amazon India. Selain itu, analisis `sizes_breakdown` digunakan untuk mengetahui ukuran produk yang paling banyak diminati konsumen pada setiap kategori.

5.4 Analisis Distribusi Status Order

Distribusi status order dianalisis menggunakan kolom `status_group` yang telah dinormalisasi. Status order dibagi menjadi empat kelompok, yaitu Shipped untuk pesanan yang berhasil dikirim, Cancelled untuk pesanan yang dibatalkan, Pending untuk pesanan yang masih dalam proses, dan Other untuk status lainnya. Dari hasil analisis, proporsi order dengan status Shipped menjadi salah satu indikator penting untuk melihat tingkat efisiensi operasional pada platform e-commerce.

Tabel 8. Analisis Distribusi Status Order

Status Group	Deskripsi	Relevansi Bisnis
Shipped	Order berhasil dikirim ke pelanggan	Indikator revenue yang terealisasi
Cancelled	Order dibatalkan sebelum dikirim	Indikator loss revenue dan customer dissatisfaction
Pending	Order masih dalam proses fulfillment	Order yang perlu monitoring lebih lanjut
Other	Status lain (refund, returned, dll.)	Memerlukan analisis lebih mendalam

5.5 Analisis Distribusi Status Order

Analisis B2B (*Business-to-Business*) dan B2C (*Business-to-Consumer*) dilakukan menggunakan kolom boolean `b2b` pada dataset. Analisis ini digunakan untuk melihat perbedaan karakteristik transaksi antara pelanggan bisnis dan konsumen individu. Beberapa aspek yang dianalisis meliputi rata-rata nilai order, kategori produk yang paling diminati, serta pola pembelian pada masing-masing jenis pelanggan.

5.6 Analisis Distribusi Status Order

Analisis geografis dilakukan berdasarkan kolom `ship_state` untuk mengetahui wilayah dengan jumlah order dan revenue tertinggi. Hasil analisis disimpan dalam file `top_states.json` dan

ditampilkan pada dashboard Streamlit dalam bentuk visualisasi interaktif. Berdasarkan hasil analisis, wilayah dengan jumlah order tertinggi umumnya berasal dari pusat ekonomi utama India, seperti Maharashtra, Karnataka, dan Tamil Nadu.

5.7 Analisis Distribusi Status Order

Berdasarkan hasil Exploratory Data Analysis (EDA), diperoleh beberapa insight utama dari dataset Amazon Sale Report, yaitu:

- a. Produk fashion wanita India, seperti kategori Set dan Kurta, menjadi kategori dengan penjualan dan revenue tertinggi di platform Amazon India
- b. Persentase order dengan status Shipped berada pada kisaran 70–75%, yang menunjukkan proses operasional pengiriman sudah cukup baik meskipun masih dapat ditingkatkan
- c. Sebagian besar transaksi berasal dari pelanggan B2C, namun transaksi B2B memiliki rata-rata nilai order yang lebih tinggi
- d. Volume order mengalami perubahan yang cukup signifikan pada beberapa bulan tertentu, yang kemungkinan dipengaruhi oleh musim belanja dan program promosi
- e. Penggunaan promosi (has_promotion) berbeda pada setiap kategori produk, menunjukkan adanya perbedaan strategi pemasaran antar kategori
- f. Persebaran order lebih banyak terkonsentrasi pada beberapa wilayah besar di India yang memiliki aktivitas ekonomi dan jumlah penduduk tinggi

BAB VI

IMPLEMENTASI PIPELINE DAN HASIL

6.1 Inisialisasi PySpark Sessions

Pipeline dimulai dengan inisialisasi SparkSession menggunakan konfigurasi yang dioptimalkan untuk stabilitas di lingkungan Windows. Konfigurasi utama meliputi alokasi memori driver dan executor masing-masing 2GB, pembatasan master ke local[2] (2 core) untuk menghindari crash worker, penonaktifan Arrow, serta pengaktifan Python worker fault handler. SparkContext juga dikonfigurasi dengan checkpoint directory untuk ketahanan pipeline, dan binding address ditetapkan ke 127.0.0.1 untuk menghindari konflik network.

Tabel 9. Inisialisasi PySpark Sessions

Parameter Konfigurasi	Nilai	Alasan
spark.driver.memory	2g	Driver dan executor masing-masing 2GB, stabil di Windows
spark.sql.shuffle.partitions	2	Mengurangi beban parallel task, stabil di Windows
spark.master	local[2]	Membatasi ke 2 core untuk stabilitas di Windows
spark.executor.memory	2g	Alokasi memori executor (sama dengan driver)
spark.sql.execution.arrow.pyspark.enabled	false	Menonaktifkan Arrow untuk menghindari crash worker
spark.python.worker.faultHandler.enabled	true	Menangkap crash Python worker secara lebih stabil
spark.driver.host	127.0.0.1	Menghindari konflik network di Windows
spark.jars	mysql-connector-j-9.7.0.jar	Mengaktifkan koneksi JDBC ke MySQL

6.2 Data Ingestion

Proses data ingestion dilakukan dengan membaca file CSV menggunakan spark.read.csv() pada PySpark dengan schema eksplisit (StructType dan 24 StructField) agar proses pembacaan data lebih cepat dibandingkan inferSchema=True. Pada tahap ini digunakan opsi multiLine=true dan escape="" untuk menangani data yang mengandung koma atau baris baru di dalam nilai kolom.

Setelah file CSV dibaca, dilakukan dua tahap pra-pemrosesan. Pertama, normalisasi format tanggal menggunakan F.coalesce() dengan beberapa format tanggal secara berurutan, yaitu MM-dd-yy, M/d/yyyy, dd-MM-yy, dan yyyy-MM-dd, sehingga kolom date langsung memiliki tipe DateType tanpa menghasilkan nilai null akibat perbedaan format. Kedua, dilakukan pembersihan kolom ship_postal_code dengan menghapus suffix .0 menggunakan regexp_replace(). Setelah proses ingestion selesai, dilakukan validasi untuk memastikan jumlah nilai null pada kolom date bernilai 0.

6.3 Data Cleaning dan Feature Engineering

Tahap data cleaning dilakukan menggunakan transformasi PySpark yang digabung dalam satu pipeline. Karena kolom date sudah bertipe DateType sejak tahap ingestion, proses ekstraksi year, month, dan month_name dapat langsung dilakukan menggunakan fungsi year(), month(), dan date_format(). Selain itu, dilakukan juga pembersihan whitespace menggunakan trim() pada beberapa kolom seperti sales_channel, ship_city, ship_state, status, dan category.

Tabel 10. Data Cleaning dan Feature Engineering

Kolom	Masalah	Jumlah Null	Solusi
row_index, unused_col	Kolom tidak bermakna	-	Drop kolom
courier_status	Banyak nilai null	6.872	Isi 'Unknown'
amount, currency	Null kritis (finansial)	7.795	Drop baris
ship_city/state/postal/country	Null geografis	33	Drop baris
fulfilled_by	Banyak nilai null	89.698	Isi 'Unknown'
promotion_ids	Banyak nilai null	49.153	Isi 'None'
sales_channel	Trailing whitespace	-	Trim()
date	Baris null pasca ingestion	0 (target)	dropna(subset=['date'])

Selain cleaning, dilakukan juga feature engineering dengan menambahkan enam kolom baru, yaitu year, month, dan month_name dari data tanggal, status_group untuk mengelompokkan status order menjadi Shipped, Cancelled, Pending, dan Other, revenue untuk menghitung total pendapatan ($qty \times amount$) yang dibulatkan menjadi dua desimal, serta has_promotion untuk menandai apakah transaksi menggunakan promosi atau tidak. Setelah seluruh proses selesai, df_clean disimpan menggunakan cache() agar proses query berikutnya menjadi lebih cepat dan efisien.

6.4 Analisis Agregat dengan Spark SQL

Analisis dilakukan dalam dua lapisan. Pertama, agregasi menggunakan PySpark DataFrame API menghasilkan DataFrame untuk penyimpanan. Kedua, Spark SQL melalui Temp View 'amazon_sales' menghasilkan 9 query analitik spesifik:

1. Q1 Top Kombinasi Kategori & Ukuran:
Produk terlaris berdasarkan kombinasi category $\times$ size pada order Shipped (TOP 15 by revenue).
2. Q2 Funnel Konversi Order
Distribusi status order dengan persentase, avg order value, jumlah B2B dan promo per status.
3. Q3 Performa Penjualan Harian
Top 10 tanggal dengan revenue tertinggi untuk analisis pola musiman atau event-driven.
4. Q4 Dampak Promosi

Perbandingan avg harga, qty, dan revenue antara order berpromo vs tidak berpromo (Shipped only).

5. Q5 Performa Channel & Fulfillment
Revenue dan order share per kombinasi sales_channel × fulfilment method.
6. Q6 Month-over-Month Growth
Pertumbuhan revenue bulanan menggunakan Window Function LAG() dengan kalkulasi persen MoM.
7. Q7 State Performance
Top 15 negara bagian berdasarkan revenue, dengan pct_orders, pct_revenue, dan revenue_rank (RANK Window).
8. Q8 Segmentasi Harga per Kategori
Pengelompokan produk ke Budget (<300 INR), Mid-Range (300-700 INR), Premium (>700 INR).
9. Q9 Courier Status Impact
Analisis pengaruh status kurir terhadap distribusi order, states covered, dan categories covered.

6.5 Penyimpanan ke MySQL

Data hasil pengolahan disimpan ke MySQL menggunakan dua metode yang disesuaikan dengan ukuran data. Tabel orders yang berisi sekitar 128 ribu data transaksi disimpan menggunakan spark.write.format('jdbc') agar proses penulisan data dapat dilakukan secara paralel dan lebih efisien. Sementara itu, tabel monthly_summary dan category_stats yang berisi data agregat dalam jumlah kecil disimpan menggunakan mysql-connector-python dengan metode batch insert karena lebih sederhana dan cepat untuk data berukuran kecil.

Tabel 11. Penyimpanan ke MySQL

Tabel MySQL	Isi Data	Metode Penyimpanan
orders	Seluruh data transaksi	Spark JDBC write (paralel)
monthly_summary	Data agregat bulanan	mysql-connector-python batch insert
category_stats	Statistik per kategori produk	mysql-connector-python batch insert

6.6 Penyimpanan ke MongoDB

Data hasil pengolahan disimpan ke MongoDB menggunakan PyMongo ke dalam dua koleksi utama. Proses penyimpanan dilakukan secara batch dengan jumlah 5.000 dokumen setiap proses untuk menghindari timeout pada data berukuran besar. Sebelum disimpan, data transaksi terlebih dahulu ditransformasi, seperti mengubah field date menjadi string agar dapat disimpan dalam format JSON, serta mengubah promotion_ids dari string menjadi list Python (array) agar sesuai dengan struktur data MongoDB.

Tabel 12. Penyimpanan ke MongoDB

Koleksi MongoDB	Isi Data	Fitur Khusus
orders_detail	Dokumen lengkap per order	promotion_ids disimpan sebagai array dan memiliki field source
category_insights	Insight agregat per kategori	Memiliki nested object price_range dan array sizes_breakdown

6.7 Integrasi Cross-System (JOIN MySQL x MongoDB)

Integrasi lintas sistem menjadi salah satu bagian penting dalam pipeline ini karena menggabungkan data dari MySQL dan MongoDB. Integrasi lintas sistem dilakukan dalam dua tahap. Pertama, data MySQL orders (filtered status=Shipped) dan data MongoDB category_insights masing-masing diambil ke Pandas DataFrame. Join kemudian dilakukan menggunakan pandas.merge() pada key 'category', lalu kolom derived price_vs_category_avg dan pct_of_cat_revenue dihitung dengan apply(lambda). Hasil diurutkan berdasarkan order_id untuk konsistensi dan ditampilkan 22 baris pertama.

Untuk query segmentasi harga, data MySQL diambil langsung via mysql.connector (pd.read_sql) dan MongoDB via PyMongo, kemudian di-merge secara Pandas. Pendekatan ini dipilih karena terbukti lebih stabil di lingkungan Windows dibandingkan Spark DataFrame JOIN yang rentan crash akibat keterbatasan worker thread.

Tabel 13. Integrasi Cross-System

Metode Integrasi	Deskripsi	Status
pandas.merge()	MongoDB → Pandas → merge di driver (tidak scalable)	Digunakan (stabil windows)
spark.createDataFrame()	MongoDB → Python list → Spark DF → Spark JOIN	Digunakan
MongoDB Spark Connector	Native connector via JAR (paling ideal)	Opsional (butuh jar tambahan)

6.8 Dashboard Streamlit

Pipeline diakhiri dengan export semua data analitik ke file JSON di folder streamlit_data/. File JSON ini kemudian dibaca oleh dashboard Streamlit (app.py) untuk divisualisasikan. Dashboard dirancang dengan tampilan dark-mode modern menggunakan custom CSS dengan palet warna GitHub-inspired.

Fitur visualisasi yang disediakan dashboard meliputi KPI Cards (Total Revenue, Total Order, Shipped Rate, Top Category), Line Chart tren bulanan interaktif, Bar Chart revenue per kategori, Pie Charts distribusi status dan B2B/B2C, Heatmap Kategori × Bulan, tabel top state dengan filter, serta tabel detail segmentasi harga hasil cross-system JOIN.

6.9 Ringkasan Hasil

Tabel 14. Ringkasan Hasil

Aspek Pipeline	Hasil
Data Diproses	128.976 baris transaksi Amazon India
Database MySQL	final bidat — 3 tabel: orders, monthly_summary, category_stats
Database MongoDB	final bidat — 2 koleksi: orders_detail, category_insights
File JSON Export	8 file JSON untuk dashboard Streamlit
Spark Query Analitik	9 query SQL (Q1–Q9) via Temp View 'amazon_sales'

Aspek Pipeline	Hasil
Integrasi Cross-System	MySQL × MongoDB via pandas.merge() (stabil di Windows)
Bug Fix Utama	Date parsing multi-format (coalesce), fix konversi date MongoDB
Dashboard	Streamlit interaktif dengan KPI cards, charts, heatmap, tabel
Teknologi Utama	PySpark, MySQL, MongoDB, PyMongo, Pandas, Streamlit, Plotly

Pipeline yang dibangun berhasil menunjukkan penerapan Big Data Pipeline secara lengkap, mulai dari proses pembacaan data mentah hingga penyajian insight bisnis melalui dashboard interaktif. Pada versi final (versi_esticka_fixed), dilakukan beberapa perbaikan penting, seperti normalisasi berbagai format tanggal pada tahap ingestion, peningkatan stabilitas SparkSession pada sistem Windows, perbaikan konversi data tanggal ke MongoDB, serta penambahan 9 query analitik menggunakan Spark SQL. Database final_bidat juga digunakan untuk mengintegrasikan MySQL dan MongoDB dalam satu sistem analitik yang saling terhubung.

BAB VII

KESIMPULAN

7.1 Kesimpulan

Proyek ini berhasil membangun Big Data Pipeline secara lengkap untuk mengolah dataset Amazon Sale Report yang berisi sekitar 128.000 transaksi menggunakan teknologi PySpark, MySQL, MongoDB, dan Streamlit. Berikut beberapa kesimpulan utama dari proyek yang telah dilakukan:

1. Pipeline berhasil berjalan dengan baik
Seluruh proses, mulai dari pembacaan data CSV, cleaning data, feature engineering, analisis menggunakan Spark SQL, penyimpanan data ke MySQL dan MongoDB, integrasi data antar sistem, hingga visualisasi dashboard berhasil diimplementasikan dalam satu alur yang terintegrasi
2. Penggunaan MySQL dan MongoDB saling melengkapi
MySQL digunakan untuk menyimpan data terstruktur dan proses query analitik, sedangkan MongoDB digunakan untuk menyimpan data semi-terstruktur seperti array dan nested object. Kombinasi keduanya membantu pengelolaan data e-commerce menjadi lebih fleksibel
3. Kualitas data berhasil ditingkatkan
Proses normalisasi format tanggal dan penanganan missing value berhasil menghasilkan dataset yang lebih bersih dan siap digunakan untuk analisis data
4. Spark SQL mampu menghasilkan analisis yang beragam
Query analitik yang dibuat mampu menghasilkan berbagai insight bisnis, seperti tren penjualan, pertumbuhan revenue bulanan, performa wilayah, pengaruh promosi, hingga segmentasi harga produk. Hal ini menunjukkan bahwa Spark dapat digunakan sebagai mesin analisis data dalam skala besar
5. Pipeline dapat berjalan stabil di lingkungan lokal
Konfigurasi SparkSession yang disesuaikan untuk sistem Windows berhasil membantu menjaga stabilitas proses pengolahan data sehingga pipeline dapat dijalankan dengan baik pada lingkungan lokal
6. Dashboard Streamlit mempermudah visualisasi hasil analisis
Dashboard yang dibangun mampu menampilkan hasil analisis dalam bentuk visualisasi interaktif sehingga informasi lebih mudah dipahami dan dapat digunakan untuk mendukung pengambilan keputusan bisnis

LAMPIRAN

Link Dataset:

<https://drive.google.com/file/d/1DttnlfN2MKGq0LCheK8h0c18nlpQGad5/view?usp=sharing>

Bukti Submit Jurnal:

